

CPSC 304 Project Cover Page

Milestone #: 2

Date: Feb 25, 2026

Group Number: 80

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Boyi Zhang	57779811	f7r8h	zboyi0402@gmail.com
Runtan Zhang	51443323	t1y3u	zhangruntan@gmail.com
Xinghao Huang	81848509	u6u0n	xinghaohuang3@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

Project Milestone 2 - Logical Design, RS, SQL DDL, Normalization, Query Design

Topic: Urban Bike Infrastructure Maintenance System

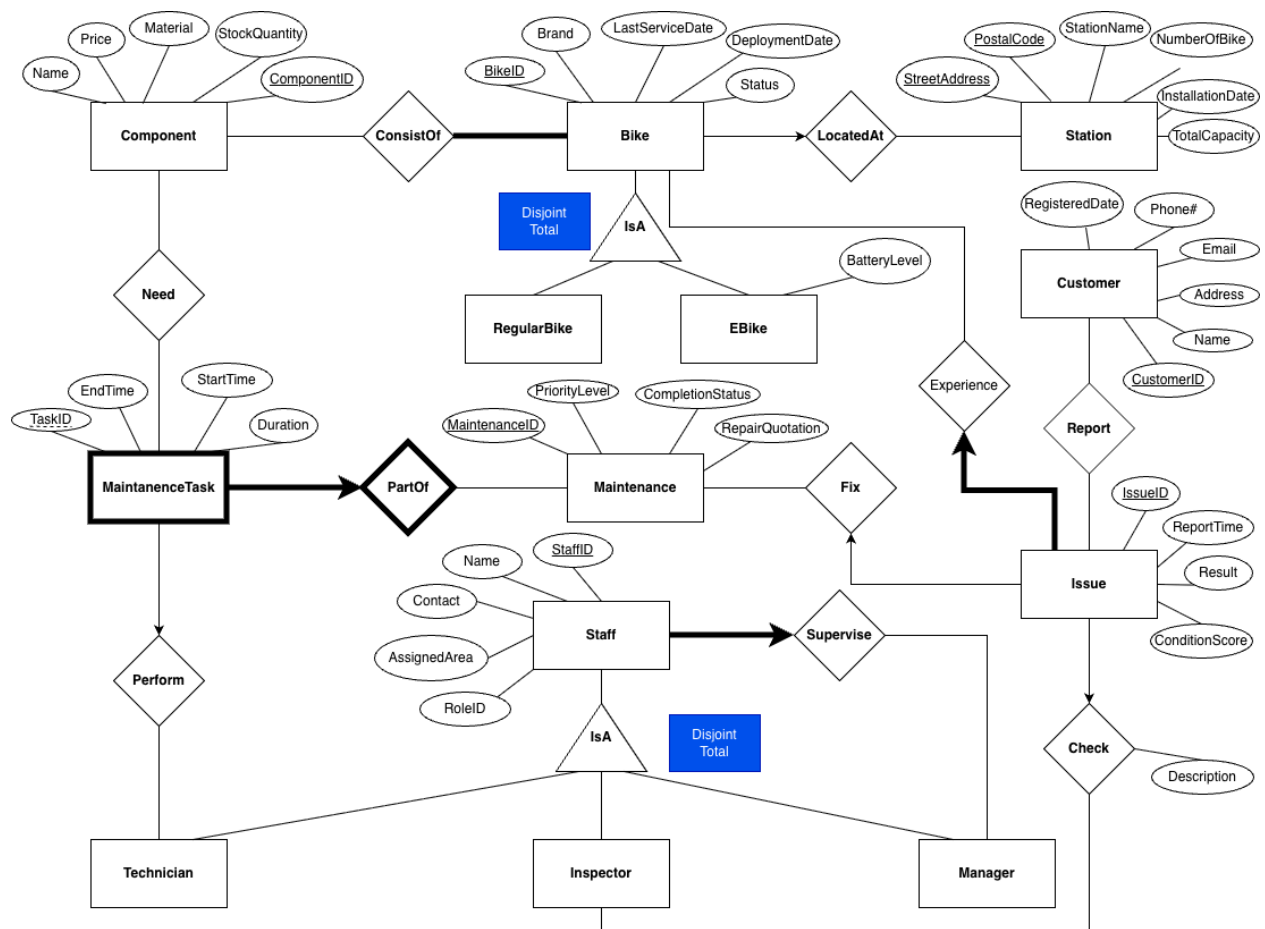
Project Summary

Our project is about urban bike infrastructure maintenance and operations. It concerns the real-world processes involved in operating and maintaining a city-wide shared bicycle system, including ensuring that bicycles deployed across urban stations remain safe, functional, and available for public use. This domain involves coordination between physical assets, maintenance staff, and operational policies to support reliable and efficient urban transportation.

Database and Programming Language for the Project

We are going to use JavaScript and the Oracle database for our project.

ER Diagram



Changes:

- Component – Bike (ConsistOf)
 - Previously: many-to-one
 - Now: many-to-many
- Component – MaintenanceTask (Need)
 - Previously: many-to-one
 - Now: many-to-many
- Primary key of Station
 - Previously: (StreetAddress, PostalCode, StationName)
 - Now: (StreetAddress, PostalCode)
- Rename EBike entity
 - Previously: E-Bike
 - Now: EBike

Note: We changed both relationships from many-to-one to many-to-many because the previous design forced each component instance to belong to exactly one bike and one maintenance

task, which is unrealistic. In practice, components represent inventory types that can be used by multiple bikes and required by multiple maintenance tasks. The updated many-to-many relationships better reflect real-world usage and remove unnecessary existence dependencies. We also updated the primary key of the Station entity to better reflect the functional dependencies and improve the overall schema design. E-Bike has been renamed to EBike to avoid errors in SQL syntax.

Schema Derived from ER Diagram

Component

Schema:

- Component(ComponentID: CHAR(4), StockQuantity: INTEGER, Material: VARCHAR(10), Price: FLOAT, Name: VARCHAR(20))

Description:

- Represents a component type in inventory, storing its identifying information and available stock quantity, which can be used by multiple bikes and required by multiple maintenance tasks.

Primary key:

- ComponentID

Candidate keys:

- Name

Foreign keys:

- None

Other Constraints:

- Name UNIQUE
- StockQuantity ≥ 0
- Price ≥ 0

ConsistOf

Schema:

- ConsistOf(ComponentID: CHAR(4), BikeID: CHAR(4))

Description:

- The ConsistOf relation represents the many-to-many relationship between Component and Bike. It records which components are part of a bike.

Primary key:

- (ComponentID, BikeID)

Candidate keys:

- None (other than primary key)

Foreign keys:

- **ComponentID**: Referencing the ComponentID attribute of the Component entity.
- **BikeID**: Referencing the BikeID attribute of the Bike entity.

Other Constraints:

- The total participation of Bike in ConsistOf cannot be enforced using standard foreign key or NOT NULL constraints. Therefore, this constraint would require an assertion to ensure that every Bike appears in at least one tuple of ConsistOf.

Need

Schema:

- Need(**ComponentID**: CHAR(4), **TaskID**: CHAR(4), **MaintenancelD**: CHAR(4))

Description:

- The Need relation represents the many-to-many relationship between Component and MaintenanceTask. It records which components are needed for a maintenance task.

Primary key:

- (ComponentID, TaskID, MaintenancelD)

Candidate keys:

- None (other than primary key)

Foreign keys:

- **ComponentID**: Referencing the ComponentID attribute of the Component entity.
- **(TaskID, MaintenancelD)**: referencing MaintenanceTask(TaskID, MaintenancelD).

Other Constraints:

- None

Bike

Schema:

- Bike(BikeID: CHAR(4), Brand: VARCHAR(10), LastServiceDate: DATE, DeploymentDate: DATE, Status: VARCHAR(10), **StreetAddress**: VARCHAR(100), **PostalCode**: CHAR(6), **StationName**: VARCHAR(20))

Description:

- Represents an individual bike in the system, storing its identifying and operational information.
- The relationship, LocatedAt, is merged with the Bike entity because of a many-to-one relationship.

Primary key:

- BikeID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **StreetAddress & PostalCode & StationName**: a composite foreign key associating with the LocatedAt relationship and referencing the composite primary key (StreetAddress, PostalCode, StationName) of the Station entity.

Other Constraints:

- Status NOT NULL
- The ISA hierarchy between Bike and its subtypes (RegularBike and E-Bike) is disjoint and total. Therefore, each Bike instance must appear in exactly one subtype relation, and the subtype relations are mutually exclusive.

Station

Schema:

- Station(StreetAddress: VARCHAR(100), PostalCode: CHAR(6), StationName: VARCHAR(20), NumberOfBike: INTEGER, InstallationDate: DATE, TotalCapacity: INTEGER)

Description:

- Represents a bike station in the system, and stores location information and capacity details.

Primary key:

- (StreetAddress, PostalCode)

Candidate keys:

- None (other than primary key)

Foreign keys:

- None

Other Constraints:

- NumberOfBike ≥ 0
- TotalCapacity ≥ 0
- StationName NOT NULL

RegularBike

Schema:

- RegularBike(**BikeID**: CHAR(4))

Description:

- RegularBike is a subtype of Bike under a disjoint and total specialization. It represents standard bikes that do not contain additional subtype-specific attributes.

Primary key:

- BikeID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **BikeID**: associating with the IsA relationship and referencing the BikeID attribute of the Bike entity

Other Constraints:

- None

EBike

Schema:

- E Bike(**BikeID**: CHAR(4), BatteryLevel: INTEGER)

Description:

- E Bike is a subtype of Bike under a disjoint and total specialization. It extends Bike by adding the BatteryLevel attribute, which tracks the charge level of electric bikes.

Primary key:

- BikeID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **BikeID**: associating with the IsA relationship and referencing the BikeID attribute of the Bike entity

Other Constraints:

- $0 \leq \text{BatteryLevel} \leq 100$

Customer

Schema:

- Customer(CustomerID: CHAR(4), Name: VARCHAR(30), Address: VARCHAR(100), Email: VARCHAR(30), Phone#: VARCHAR(20), RegisteredDate: DATE)

Description:

- Represents registered users of the system. It stores personal and contact information, including registration details. CustomerID uniquely identifies each customer.

Primary key:

- CustomerID

Candidate keys:

- None

Foreign keys:

- None

Other Constraints:

- Email UNIQUE NOT NULL
- RegisteredDate NOT NULL

Report

Schema:

- Report(CustomerID: CHAR(4), IssueID: CHAR(4))

Description:

- The Report relation represents the many-to-many relationship between Customer and Issue. It records which customer reports which issue.

Primary key:

- (CustomerID, IssueID)

Candidate keys:

- None (other than primary key)

Foreign keys:

- **CustomerID**: referencing the CustomerID attribute of the Customer entity
- **IssueID**: referencing the IssueID of the Issue entity

Other Constraints:

- None

Issue

Schema:

- Issue(IssueID: CHAR(4), ReportTime: TIMESTAMP, Result: VARCHAR(20), ConditionScore: FLOAT, **BikeID**: CHAR(4), **MaintenanceID**: CHAR(4), Description: CLOB, **InspectorID**: CHAR(4))

Description:

- Represents reported problems associated with bikes.
- The relationships, Experience, Check, and Fix, are merged with the Issue entity because of many-to-one relationships.

Primary key:

- IssueID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **BikeID**: associating with the Experience relationship, referencing the BikeID attribute of the Bike entity.
- **MaintenanceID**: associating with the Fix relationship, referencing the MaintenanceID attribute of the Maintenance Entity.
- **InspectorID**: associating with the Check relationship, referencing the StaffID attribute of the Inspector entity (note that Inspector is a subtype of staff).

Other Constraints:

- Result NOT NULL
- Description NOT NULL
- ConditionScore NOT NULL
- BikeID NOT NULL
- $0.0 \leq \text{ConditionScore} \leq 5.0$

MaintenanceTask

Schema:

- MaintenanceTask(TaskID: CHAR(4), **MaintenanceID**: CHAR(4), Duration: TIME, StartTime: TIMESTAMP, EndTime: TIMESTAMP, **TechnicianID**: CHAR(4))

Description:

- MaintenanceTask is a weak entity that represents individual tasks within a maintenance record.
- The relationships, PartOf and Perform, are merged with the MaintenanceTask entity because of both many-to-one relationship and weak entity relationship.

Primary key:

- TaskID & MaintenanceID (Note that, TaskID by itself is a partial key of the weak entity MaintenanceTask)

Candidate keys:

- None (other than primary key)

Foreign keys:

- **MaintenanceID**: associating with the PartOf relationship, referencing the MaintenanceID attribute of the Maintenance entity.
- **TechnicianID**: associating with the Perform relationship, referencing the StaffID attribute of the Technician entity (note that Technician is a subtype of Staff).

Other Constraints:

- MaintenanceID NOT NULL

Maintenance

Schema:

- Maintenance(MaintenanceID: CHAR(4), PriorityLevel: INTEGER, CompletionStatus: VARCHAR(10), RepairQuotation: CLOB)

Description:

- The Maintenance entity represents a maintenance record associated with one or more issues. It stores priority level, completion status, and repair quotation information. Each maintenance record may consist of multiple maintenance tasks.

Primary key:

- MaintenanceID

Candidate keys:

- None (other than primary key)

Foreign keys:

- None

Other Constraints:

- PriorityLevel ≥ 0
- CompletionStatus NOT NULL

Staff

Schema:

- Staff(StaffID: CHAR(4), Name: VARCHAR(30), Contact: VARCHAR(20), AssignedArea: CLOB, RoleID: CHAR(4), **ManagerID**: CHAR(4))

Description:

- The Staff entity represents employees in the system.
- The relationship, Supervise, is merged with the Staff entity because of a many-to-one relationship. And since Manager is a subtype of Staff, an additional foreign key, ManagerID, is added in order to reference a manager.

Primary key:

- StaffID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **ManagerID**: associating with the Supervise relationship, referencing the StaffID attribute of the Manager entity (note that Manager is a subtype of Staff, and a manager will supervise different types of staff).

Other Constraints:

- ManagerID NOT NULL
- The ISA hierarchy between Staff and its subtypes (Technician, Inspector, Manager) is disjoint and total. Therefore, each Staff instance must appear in exactly one subtype relation, and the subtype relations are mutually exclusive.

Technician

Schema:

- Technician(**StaffID**: CHAR(4))

Description:

- Technician is a subtype of Staff under a disjoint and total specialization. It represents staff members responsible for performing maintenance tasks.

Primary key:

- StaffID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **StaffID**: associating with the IsA relationship, referencing the StaffID attribute of the Staff entity.

Other Constraints:

- None

Inspector

Schema:

- Inspector(**StaffID**: CHAR(4))

Description:

- Inspector is a subtype of Staff under a disjoint and total specialization. It represents staff members responsible for inspecting and evaluating reported issues.

Primary key:

- StaffID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **StaffID**: associating with the IsA relationship, referencing the StaffID attribute of the Staff entity.

Other Constraints:

- None

Manager

Schema:

- Manager(**StaffID**: CHAR(4))

Description:

- Manager is a subtype of Staff under a disjoint and total specialization. It represents staff members responsible for supervising other staff members.

Primary key:

- StaffID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **StaffID**: associating with the IsA relationship, referencing the StaffID attribute of the Staff entity.

Other Constraints:

- None

Functional Dependencies

Component

- ComponentID → Name, Price, Material, StockQuantity
- Name → ComponentID, Price, Material, StockQuantity

ConsistOf

- None (other than trivial ones)

Need

- None (other than trivial ones)

Bike

- BikeID → Brand, LastServiceDate, DeploymentDate, Status, StreetAddress, PostalCode, StationName
- StreetAddress, PostalCode → StationName

Station

- StreetAddress, PostalCode → StationName, InstallationDate, TotalCapacity, NumberOfBike
- StationName → InstallationDate, TotalCapacity, NumberOfBike

RegularBike

- None (other than trivial ones)

EBike

- BikeID → BatteryLevel

Customer

- CustomerID → Name, Address, Email, Phone#, RegisteredDate
- Email → Name, Phone#, Address

Report

- None (other than trivial ones)

Issue

- IssueID → ReportTime, Result, ConditionScore, BikeID, MaintenanceID, Description, InspectorID
- ConditionScore, Description → Result

MaintenanceTask

- TaskID, MaintenanceID → Duration, StartTime, EndTime, TechnicianID

Maintenance

- MaintenanceID → PriorityLevel, CompletionStatus, RepairQuotation

Staff

- StaffID → Name, Contact, AssignedArea, RoleID, ManagerID

Technician

- None (other than trivial ones)

Inspector

- None (other than trivial ones)

Manager

- None (other than trivial ones)

Normalization

We will normalize each table into **BCNF**

Component

Original Schema:

- Component(ComponentID: CHAR(4), StockQuantity: INTEGER, Material: VARCHAR(10), Price: FLOAT, Name: VARCHAR(20))

Functional Dependencies:

- ComponentID → Name, Price, Material, StockQuantity
- Name → ComponentID, Price, Material, StockQuantity

Step 1: Compute Closure:

- ComponentID⁺ = {ComponentID, Name, Price, Material, StockQuantity}
- Name⁺ = {Name, ComponentID, Price, Material, StockQuantity}

Step 2: Check BCNF

- From the given FDs and closures, both ComponentID and Name determine all attributes. Therefore, both are candidate keys (and hence superkeys).
- Since every non-trivial FD has a superkey on the left-hand side, Component is already in BCNF, and no decomposition is required.

Step 3: Decomposition

- Not required

Final Schema After Normalization:

- Component(ComponentID: CHAR(4), StockQuantity: INTEGER, Material: VARCHAR(10), Price: FLOAT, Name: VARCHAR(20))

Keys After Normalization:

- Component:
 - PK: ComponentID
 - CK: Name
 - FK: None
 - Other Constraints:
 - Name UNIQUE
 - StockQuantity ≥ 0
 - Price ≥ 0

ConsistOf

Original Schema:

- ConsistOf(ComponentID: CHAR(4), BikeID: CHAR(4))

Functional Dependencies:

- None (other than trivial FDs)

Step 1: Compute Closure

- {ComponentID, BikeID}⁺ = {ComponentID, BikeID}

Step 2: Check BCNF

- The only functional dependency in this relation is the primary key dependency. Since its left-hand side is a superkey, no FD violates BCNF. Therefore, ConsistOf is already in BCNF and no decomposition is required.

Step 3: Decomposition:

- Not required

Final Schema After Normalization:

- ConsistOf(ComponentID: CHAR(4), BikeID: CHAR(4))

Keys After Normalization:

- ConsistOf
 - PK: (ComponentID, BikeID)
 - CK: None (other than PK)
 - FK: ComponentID → Component(ComponentID); BikeID → Bike(BikeID)
 - Other Constraints:
 - The total participation of Bike in ConsistOf cannot be enforced using standard foreign key or NOT NULL constraints. Therefore, this constraint would require an assertion to ensure that every Bike appears in at least one tuple of ConsistOf.

Need

Original Schema:

- Need(ComponentID: CHAR(4), TaskID: CHAR(4), MaintenanceID: CHAR(4))

Functional Dependencies:

- None (other than trivial FDs)

Step 1: Compute Closure:

- {ComponentID, TaskID, MaintenanceID}⁺ = {ComponentID, TaskID, MaintenanceID}

Step 2: Check BCNF

- The only functional dependency in this relation is the primary key dependency. Since its left-hand side is a superkey, no FD violates BCNF. Therefore, Need is already in BCNF and no decomposition is required.

Step 3: Decomposition

- Not required

Final Schema After Normalization:

- Need(ComponentID: CHAR(4), TaskID: CHAR(4), MaintenanceID: CHAR(4))

Keys After Normalization:

- Need:
 - PK: (ComponentID, TaskID, MaintenanceID)
 - CK: None (Other than PK)
 - FK: ComponentID → Component(ComponentID); (TaskID, MaintenanceID) → MaintenanceTask(TaskID, MaintenanceID)
 - Other Constraints: None

Bike

Original Schema:

- Bike(BikeID: CHAR(4), Brand: VARCHAR(10), LastServiceDate: DATE, DeploymentDate: DATE, Status: VARCHAR(10), **StreetAddress**: VARCHAR(100), **PostalCode**: CHAR(6), **StationName**: VARCHAR(20))

Functional Dependencies:

- BikeID $\rightarrow$ Brand, LastServiceDate, DeploymentDate, Status, StreetAddress, PostalCode, StationName
- StreetAddress, PostalCode $\rightarrow$ StationName

Step 1: Compute Closure:

- BikeID⁺ = {BikeID, Brand, LastServiceDate, DeploymentDate, Status, StreetAddress, PostalCode, StationName}
- {StreetAddress, PostalCode}⁺ = {StreetAddress, PostalCode, StationName}
- BikeID is the only key

Step 2: Check BCNF:

- BikeID is a superkey that determines all the other attribute $\rightarrow$ OK
- (StreetAddress, PostalCode) is not a superkey of Bike, yet it determines StationName. It violates BCNF $\rightarrow$ Decompose

Step 3: Decomposition:

- Decompose on StreetAddress, PostalCode $\rightarrow$ StationName
 - BikeStation(StreetAddress, PostalCode, StationName)
 - Bike(BikeID, Brand, LastServiceDate, DeploymentDate, Status, StreetAddress, PostalCode)
- Check BikeStation and Bike in BCNF
 - BikeStation: StreetAddress, PostalCode $\rightarrow$ StationName is a superkey $\rightarrow$ done.
 - Bike: BikeID $\rightarrow$ Brand, LastServiceDate, DeploymentDate, Status, StreetAddress, PostalCode is a superkey $\rightarrow$ done

Final Schema After Normalization:

- BikeStation(StreetAddress: VARCHAR(100), PostalCode: CHAR(6), StationName: VARCHAR(20))
- Bike(BikeID: CHAR(4), Brand: VARCHAR(10), LastServiceDate: DATE, DeploymentDate: DATE, Status: VARCHAR(10), **StreetAddress**: VARCHAR(100), **PostalCode**: CHAR(6))

Keys After Normalization:

- BikeStation:
 - PK: (StreetAddress, PostalCode)
 - CK: None (other than PK)
 - FK: None
 - Other Constraints:
 - StationName NOT NULL
- Bike
 - PK: BikeID
 - CK: None (other than PK)
 - FK: (StreetAddress, PostalCode) $\rightarrow$ BikeStation(StreetAddress, PostalCode)

- Other Constraints:
 - Status NOT NULL
 - The ISA hierarchy between Bike and its subtypes (RegularBike and E-Bike) is disjoint and total. Therefore, each Bike instance must appear in exactly one subtype relation, and the subtype relations are mutually exclusive.

RegularBike

Original Schema:

- RegularBike(**BikeID**: CHAR(4))

Functional Dependencies:

- None (other than trivial FDs)

Step 1: Compute Closure:

- $\text{BikeID}^+ = \{\text{BikeID}\}$

Step 2: Check BCNF

- The only functional dependency in this relation is the primary key dependency. Since its left-hand side is a superkey, no FD violates BCNF. Therefore, RegularBike is already in BCNF and no decomposition is required.

Step 3: Decomposition

- Not required

Final Schema After Normalization:

- RegularBike(**BikeID**: CHAR(4))

Keys After Normalization:

- RegularBike
 - PK: BikeID
 - CK: None (other than BikeID)
 - FK: $\text{BikeID} \rightarrow \text{Bike}(\text{BikeID})$
 - Other Constraints:
 - None

EBike

Original Schema:

- EBike(**BikeID**: CHAR(4), BatteryLevel: INTEGER)

Functional Dependencies:

- $\text{BikeID} \rightarrow \text{BatteryLevel}$

Step 1: Compute Closure:

- $\text{BikeID}^+ = \{\text{BikeID}, \text{BatteryLevel}\}$

Step 2: Check BCNF

- From the given FDs and closures, BikeID determines all attributes. Therefore, it is a candidate key (and hence a superkey).
- Since every non-trivial FD has a superkey on the left-hand side, E-Bike is already in BCNF, and no decomposition is required.

Step 3: Decomposition

- Not required

Final Schema After Normalization:

- E Bike(**BikeID**: CHAR(4), BatteryLevel: INTEGER)

Keys After Normalization:

- E Bike
 - PK: BikeID
 - CK: None (other than BikeID)
 - FK: BikeID → Bike(BikeID)
 - Other Constraints:
 - $0 \leq \text{BatteryLevel} \leq 100$

Station

Original Schema:

- Station(StreetAddress: VARCHAR(100), PostalCode: CHAR(6), StationName: VARCHAR(20), NumberOfBike: INTEGER, InstallationDate: DATE, TotalCapacity: INTEGER)

Functional Dependencies:

- StationName → InstallationDate, TotalCapacity, NumberOfBike
- StreetAddress, PostalCode → StationName, InstallationDate, TotalCapacity, NumberOfBike

Step 1: Compute Closure:

- {StreetAddress, PostalCode}⁺ = {StreetAddress, PostalCode, StationName, NumberOfBike, InstallationDate, TotalCapacity}
- {StationName}⁺ = StationName, InstallationDate, TotalCapacity, NumberOfBike
- (StreetAddress, PostalCode) is the only key

Step 2: Check BCNF:

- (StreetAddress, PostalCode) is the minimal key that determines all the other attributes → OK
- StationName is not a superkey of Station, yet it determines InstallationDate, TotalCapacity, and NumberOfBike. It violates BCNF → Decompose

Step 3: Decomposition:

- Decompose on StationName → InstallationDate, TotalCapacity, NumberOfBike
 - StationInfo(StationName, InstallationDate, TotalCapacity, NumberOfBike)
 - StationAddress(StreetAddress, PostalCode, StationName)
- Check StationInfo and StationAddress in BCNF
 - StationInfo: StationName → InstallationDate, TotalCapacity, NumberOfBike is a superkey → done.
 - StationAddress: StreetAddress, PostalCode → StationName → done

Final Schema After Normalization:

- StationInfo(StationName: VARCHAR(20), NumberOfBike: INTEGER, InstallationDate: DATE, TotalCapacity: INTEGER)
- StationAddress(StreetAddress: VARCHAR(100), PostalCode: CHAR(6), **StationName**: VARCHAR(20))

Keys After Normalization:

- StationInfo:
 - PK: StationName
 - CK: None (other than PK)
 - FK: None
 - Other Constraints:
 - $\text{NumberOfBike} \geq 0$
 - $\text{TotalCapacity} \geq 0$
- StationAddress
 - PK: (StreetAddress, PostalCode)
 - CK: None (other than PK)
 - FK: StationName → StationInfo(StationName)
 - Other Constraints:
 - StationName NOT NULL

Customer

Original Schema:

- Customer(CustomerID: CHAR(4), Name: VARCHAR(30), Address: VARCHAR(100), Email: VARCHAR(30), Phone#: VARCHAR(20), RegisteredDate: DATE)

Functional Dependencies:

- CustomerID → Name, Address, Email, Phone#, RegisteredDate
- Email → Name, Phone#, Address

Step 1: Compute Closure:

- CustomerID⁺ = {CustomerID, Name, Address, Email, Phone#, RegisteredDate}
- Email⁺ = {Email, Name, Phone#, Address}
- CustomerID is the only key

Step 2: Check BCNF:

- CustomerID is a superkey that determines all the other attributes → OK
- Email is not a superkey of Customer, yet it determines Name, Phone#, and Address. It violates BCNF → Decompose

Step 3: Decomposition:

- Decompose on Email → Name, Phone#, Address
 - CustomerContact(Email, Name, Phone#, Address)
 - CustomerAccount(CustomerID, Email, RegisteredDate)
- Check CustomerContact and CustomerAccount in BCNF
 - CustomerContact: Email → Name, Phone#, Address is a superkey → done.
 - CustomerAccount: CustomerID → Email, RegisteredDate is a superkey → done

Final Schema After Normalization:

- CustomerContact(Email: VARCHAR(30), Name: VARCHAR(30), Address: VARCHAR(100), Phone#: VARCHAR(20))
- CustomerAccount(CustomerID: CHAR(4), **Email**: VARCHAR(30), RegisteredDate: DATE)

Keys After Normalization:

- CustomerContact:

- PK: Email
- CK: None (other than PK)
- FK: None
- Other Constraints:
 - None
- CustomerAccount:
 - PK: CustomerID
 - CK: None (other than PK)
 - FK: Email → CustomerContact(Email)
 - Other Constraints:
 - Email UNIQUE NOT NULL
 - RegisteredDate NOT NULL

Report

Original Schema:

- Report(CustomerID: CHAR(4), IssueID: CHAR(4))

Functional Dependencies:

- None (other than trivial FDs)

Step 1: Compute Closure

- {CustomerID, IssueID}⁺ = {CustomerID, IssueID}

Step 2: Check BCNF

- The only functional dependency in this relation is the primary key dependency. Since its left-hand side is a superkey, no FD violates BCNF. Therefore, Report is already in BCNF and no decomposition is required.

Step 3: Decomposition

- Not required

Final Schema After Normalization:

- Report(CustomerID: CHAR(4), IssueID: CHAR(4))

Keys After Normalization:

- Report:
 - PK: (CustomerID, IssueID)
 - CK: None (other PK)
 - FK: CustomerID → CustomerAccount(CustomerID); IssueID → IssueRecord(IssueID)
 - Other Constraints:
 - None

Issue

Original Schema:

- Issue(IssueID: CHAR(4), ReportTime: TIMESTAMP, Result: VARCHAR(20), ConditionScore: FLOAT, **BikeID**: CHAR(4), **MaintenanceID**: CHAR(4), Description: CLOB, **InspectorID**: CHAR(4))

Functional Dependencies:

- IssueID $\rightarrow$ ReportTime, Result, ConditionScore, BikeID, MaintenanceID, Description, InspectorID
- ConditionScore, Description $\rightarrow$ Result

Step 1: Compute Closure:

- IssueID⁺ = {IssueID, ReportTime, Result, ConditionScore, BikeID, MaintenanceID, Description, InspectorID}
- {ConditionScore, Description}⁺ = {ConditionScore, Description, Result}
- IssueID is the only key

Step 2: Check BCNF:

- IssueID is a superkey that determines all the other attributes $\rightarrow$ OK
- (ConditionScore, Description) is not a superkey of Issue, yet it determines Result. It violates BCNF $\rightarrow$ Decompose

Step 3: Decomposition:

- Decompose on ConditionScore, Description $\rightarrow$ Result
 - IssueRule(ConditionScore, Description, Result)
 - IssueRecord(IssueID, ReportTime, ConditionScore, BikeID, MaintenanceID, Description, InspectorID)
- Check IssueRule and IssueRecord in BCNF
 - IssueRule: (ConditionScore, Description) $\rightarrow$ Result is a superkey $\rightarrow$ done.
 - IssueRecord: IssueID $\rightarrow$ ReportTime, ConditionScore, BikeID, MaintenanceID, Description, InspectorID is a superkey $\rightarrow$ done

Final Schema After Normalization:

- IssueRule(ConditionScore: FLOAT, Description: CLOB, Result: VARCHAR(20))
- IssueRecord(IssueID: CHAR(4), ReportTime: TIMESTAMP, **ConditionScore**: FLOAT, **Description**: CLOB, **BikeID**: CHAR(4), **MaintenanceID**: CHAR(4), **InspectorID**: CHAR(4))

Keys After Normalization:

- IssueRule:
 - PK: (ConditionScore, Description)
 - CK: None (other than PK)
 - FK: None
 - Other Constraints:
 - Result NOT NULL
- IssueRecord:
 - PK: IssueID
 - CK: None (other than PK)
 - FK:
 - (ConditionScore, Description) $\rightarrow$ IssueRule(ConditionScore, Description);
 - BikeID $\rightarrow$ Bike(BikeID);
 - MaintenanceID $\rightarrow$ Maintenance(MaintenanceID);

- InspectorID → Inspector(StaffID)
- Other Constraints:
 - Description NOT NULL
 - ConditionScore NOT NULL
 - BikeID NOT NULL
 - $0.0 \leq \text{ConditionScore} \leq 5.0$

MaintenanceTask

Original Schema:

- MaintenanceTask(TaskID: CHAR(4), **MaintenanceID**: CHAR(4), Duration: TIME, StartTime: TIMESTAMP, EndTime: TIMESTAMP, **TechnicianID**: CHAR(4))

Functional Dependencies:

- TaskID, MaintenanceID → Duration, StartTime, EndTime, TechnicianID

Step 1: Compute Closure:

- $\{\text{TaskID}, \text{MaintenanceID}\}^+ = \{\text{TaskID}, \text{MaintenanceID}, \text{Duration}, \text{StartTime}, \text{EndTime}, \text{TechnicianID}\}$

Step 2: Check BCNF:

- From the given FDs, (TaskID, MaintenanceID) determines all attributes, and it is the primary key. Hence, it is a superkey.
- Since every non-trivial FD has a superkey on the left-hand side, MaintenanceTask is already in BCNF, and no decomposition is required.

Step 3: Decomposition:

- Not required

Final Schema After Normalization:

- MaintenanceTask(TaskID: CHAR(4), **MaintenanceID**: CHAR(4), Duration: TIME, StartTime: TIMESTAMP, EndTime: TIMESTAMP, **TechnicianID**: CHAR(4))

Keys After Normalization:

- MaintenanceTask:
 - PK: (TaskID, MaintenanceID)
 - CK: None (other than PK)
 - FK:
 - MaintenanceID → Maintenance(MaintenanceID)
 - TechnicianID → Technician(StaffID)
 - Other Constraints:
 - MaintenanceID NOT NULL

Maintenance

Original Schema:

- Maintenance(**MaintenanceID**: CHAR(4), PriorityLevel: INTEGER, CompletionStatus: VARCHAR(10), RepairQuotation: CLOB)

Functional Dependencies:

- MaintenanceID → PriorityLevel, CompletionStatus, RepairQuotation

Step 1: Compute Closure

- $MaintenanceID^+ = \{MaintenanceID, PriorityLevel, CompletionStatus, RepairQuotation\}$

Step 2: Check BCNF

- From the given FDs, MaintenanceID determines all attributes, and it is the primary key. Hence, it is a superkey.
- Since every non-trivial FD has a superkey on the left-hand side, Maintenance is already in BCNF, and no decomposition is required.

Step 3: Decomposition

- Not required

Final Schema After Normalization:

- Maintenance(MaintenanceID: CHAR(4), PriorityLevel: INTEGER, CompletionStatus: VARCHAR(10), RepairQuotation: CLOB)

Keys After Normalization:

- Maintenance:
 - PK: MaintenanceID
 - CK: None (other than PK)
 - FK: None
 - Other Constraints:
 - $PriorityLevel \geq 0$
 - CompletionStatus NOT NULL

Staff

Original Schema:

- Staff(StaffID: CHAR(4), Name: VARCHAR(30), Contact: VARCHAR(20), AssignedArea: CLOB, RoleID: CHAR(4), **ManagerID**: CHAR(4))

Functional Dependencies:

- $StaffID \rightarrow Name, Contact, AssignedArea, RoleID, ManagerID$

Step 1: Compute Closure

- $StaffID^+ = \{StaffID, Name, Contact, AssignedArea, RoleID, ManagerID\}$

Step2: Check BCNF:

- From the given FDs, StaffID determines all attributes, and it is the primary key. Hence, it is a superkey.
- Since every non-trivial FD has a superkey on the left-hand side, Staff is already in BCNF, and no decomposition is required.

Step3: Decomposition:

- Not required

Final Schema After Normalization:

- Staff(StaffID: CHAR(4), Name: VARCHAR(30), Contact: VARCHAR(20), AssignedArea: CLOB, RoleID: CHAR(4), **ManagerID**: CHAR(4))

Keys After Normalization:

- Staff:
 - PK: StaffID
 - CK: None
 - FK: ManagerID $\rightarrow$ Manager(StaffID)
 - Other Constraints:

- ManagerID NOT NULL
- The ISA hierarchy between Staff and its subtypes (Technician, Inspector, Manager) is disjoint and total. Therefore, each Staff instance must appear in exactly one subtype relation, and the subtype relations are mutually exclusive.

Technician

Original Schema:

- Technician(**StaffID**: CHAR(4))

Functional Dependencies:

- None (other than trivial ones)

Step 1: Compute Closure

- $\text{StaffID}^+ = \{\text{StaffID}\}$

Step2: Check BCNF:

- The only functional dependency in this relation is the primary key dependency. Since its left-hand side is a superkey, no FD violates BCNF. Therefore, Technician is already in BCNF and no decomposition is required.

Step3: Decomposition:

- Not required

Final Schema After Normalization:

- Technician(**StaffID**: CHAR(4))

Keys After Normalization:

- Manager:
 - PK: StaffID
 - CK: None
 - FK: StaffID $\rightarrow$ Staff(StaffID)
 - Other Constraints:
 - None

Inspector

Original Schema:

- Inspector(**StaffID**: CHAR(4))

Functional Dependencies:

- None (other than trivial ones)

Step 1: Compute Closure

- $\text{StaffID}^+ = \{\text{StaffID}\}$

Step2: Check BCNF:

- The only functional dependency in this relation is the primary key dependency. Since its left-hand side is a superkey, no FD violates BCNF. Therefore, Inspector is already in BCNF and no decomposition is required.

Step3: Decomposition:

- Not required

Final Schema After Normalization:

- Inspector(**StaffID**: CHAR(4))

Keys After Normalization:

- Manager:
 - PK: StaffID
 - CK: None
 - FK: StaffID → Staff(StaffID)
 - Other Constraints:
 - None

Manager

Original Schema:

- Manager(**StaffID**: CHAR(4))

Functional Dependencies:

- None (other than trivial ones)

Step 1: Compute Closure

- StaffID⁺ = {StaffID}

Step2: Check BCNF:

- The only functional dependency in this relation is the primary key dependency. Since its left-hand side is a superkey, no FD violates BCNF. Therefore, Manager is already in BCNF and no decomposition is required.

Step3: Decomposition:

- Not required

Final Schema After Normalization:

- Manager(**StaffID**: CHAR(4))

Keys After Normalization:

- Manager:
 - PK: StaffID
 - CK: None
 - FK: StaffID → Staff(StaffID)
 - Other Constraints:
 - None

SQL DDL Statements

Component

```
CREATE TABLE Component (  
    ComponentID CHAR(4),  
    StockQuantity INTEGER CHECK (StockQuantity >= 0),  
    Material VARCHAR(10),  
    Price FLOAT CHECK (Price >= 0),  
    Name VARCHAR(20) UNIQUE,  
    PRIMARY KEY (ComponentID)  
);
```

ConsistOf

```
CREATE TABLE ConsistOf (  
    ComponentID CHAR(4),  
    BikeID CHAR(4),  
    PRIMARY KEY (ComponentID, BikeID),  
    FOREIGN KEY (ComponentID) REFERENCES Component(ComponentID)  
        ON DELETE CASCADE,  
    FOREIGN KEY (BikeID) REFERENCES Bike(BikeID)  
        ON DELETE CASCADE  
);
```

Justification:

- We add ON DELETE CASCADE for both two foreign keys because it is a pure many-to-many relationship table that has no independent existence.
- If either a bike or a component is deleted, the corresponding relationship tuples in ConsistOf should also be removed to maintain referential integrity.

Need

```
CREATE TABLE Need (  
    ComponentID CHAR(4),  
    TaskID CHAR(4),  
    MaintenanceID CHAR(4),  
    PRIMARY KEY (ComponentID, TaskID, MaintenanceID),  
    FOREIGN KEY (ComponentID) REFERENCES Component(ComponentID)  
        ON DELETE CASCADE,  
    FOREIGN KEY (TaskID, MaintenanceID) REFERENCES  
MaintenanceTask(TaskID,MaintenanceID)  
        ON DELETE CASCADE  
);
```

Justification:

- We add ON DELETE CASCADE for both two foreign keys because it is a pure many-to-many relationship table that has no independent existence.
- If either a component or a task is deleted, the corresponding relationship tuples in Need should also be removed to maintain referential integrity.

BikeStation (decomposed from the original Bike entity)

```
CREATE TABLE BikeStation (  
    StreetAddress VARCHAR(100),  
    PostalCode CHAR(6),  
    StationName VARCHAR(20) NOT NULL,  
    PRIMARY KEY(StreetAddress, PostalCode)  
);
```

Bike (decomposed from the original Bike entity)

```
CREATE TABLE Bike (  
    BikeID CHAR(4),  
    Brand VARCHAR(10),  
    LastServiceDate DATE,  
    DeploymentDate DATE,  
    Status VARCHAR(10) NOT NULL,  
    StreetAddress VARCHAR(100),  
    PostalCode CHAR(6),  
    PRIMARY KEY (BikeID),  
    FOREIGN KEY (StreetAddress, PostalCode) REFERENCES BikeStation(StreetAddress,  
    PostalCode)  
        ON UPDATE CASCADE  
);
```

Justification:

- We add ON UPDATE CASCADE for the foreign key (StreetAddress, PostalCode) because cascading the update ensures that all referencing Bike tuples remain consistent with the updated station information.

RegularBike

```
CREATE TABLE RegularBike (  
    BikeID CHAR(4),  
    PRIMARY KEY (BikeID),  
    FOREIGN KEY (BikeID) REFERENCES Bike(BikeID)  
        ON DELETE CASCADE  
);
```

Justification:

- RegularBike is a subtype of Bike, so we add ON DELETE CASCADE to ensure referential integrity because a subtype tuple cannot exist without the existence of its corresponding supertype tuple.

EBike

```
CREATE TABLE EBike (  
    BikeID CHAR(4),  
    BatteryLevel INTEGER CHECK (BatteryLevel BETWEEN 0 AND 100),  
    PRIMARY KEY (BikeID),  
    FOREIGN KEY (BikeID) REFERENCES Bike(BikeID)  
        ON DELETE CASCADE  
);
```

Justification:

- EBike is a subtype of Bike, so we add ON DELETE CASCADE to ensure referential integrity because a subtype tuple cannot exist without the existence of its corresponding supertype tuple.

StationInfo (Decomposed from Station)

```
CREATE TABLE StationInfo (  
    StationName VARCHAR(20),  
    NumberOfBike INTEGER CHECK (NumberOfBike >= 0),  
    InstallationDate DATE,  
    TotalCapacity INTEGER CHECK (TotalCapacity >= 0),  
    PRIMARY KEY (StationName)  
);
```

StationAddress (Decomposed from Station)

```
CREATE TABLE StationAddress (  
    StreetAddress VARCHAR(100),  
    PostalCode CHAR(6),  
    StationName VARCHAR(20) NOT NULL,  
    PRIMARY KEY (StreetAddress, PostalCode),  
    FOREIGN KEY (StationName) REFERENCES StationInfo(StationName)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

Justification:

- We add ON DELETE CASCADE because StationAddress depends on StationInfo. If a station is removed from StationInfo, its corresponding record at StationAddress should also be removed.
- We add ON UPDATE CASCADE to ensure the changes to StationName are automatically propagated to StationAddress.

CustomerContact (decomposed from Customer)

```
CREATE TABLE CustomerContact (
    Email VARCHAR(30),
    Name VARCHAR(30),
    Address VARCHAR(100),
    PhoneNumber VARCHAR(20),
    PRIMARY KEY (Email)
);
```

CustomerAccount (decomposed from Customer)

```
CREATE TABLE CustomerAccount (
    CustomerID CHAR(4),
    Email VARCHAR(30) UNIQUE NOT NULL,
    RegisteredDate DATE NOT NULL,
    PRIMARY KEY (CustomerID),
    FOREIGN KEY (Email) REFERENCES CustomerContact(Email)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Justification:

- We add ON DELETE CASCADE because CustomerAccount depends on CustomerInfo. If a contact information is removed from CustomerInfo, its corresponding record at CustomerAccount should also be removed.
- We add ON UPDATE CASCADE to ensure the changes to Email are automatically propagated to CustomerAccount.

Report

```
CREATE TABLE Report (
    CustomerID CHAR(4),
    IssueID CHAR(4),
    PRIMARY KEY (CustomerID, IssueID),
    FOREIGN KEY (CustomerID) REFERENCES CustomerAccount(CustomerID)
        ON DELETE CASCADE,
    FOREIGN KEY (IssueID) REFERENCES IssueRecord(IssueID)
```

ON DELETE CASCADE

);

Justification:

- We add ON DELETE CASCADE for both two foreign keys because it is a pure many-to-many relationship table that has no independent existence.
- If either a customer or an issue is deleted, the corresponding relationship tuples in Report should also be removed to maintain referential integrity.

IssueRule (decomposed from Issue)

```
CREATE TABLE IssueRule (  
    ConditionScore FLOAT,  
    Description CLOB,  
    Result VARCHAR2(20) NOT NULL,  
    PRIMARY KEY (ConditionScore, Description)  
);
```

IssueRecord (decomposed from Issue)

```
CREATE TABLE IssueRecord (  
    IssueID CHAR(4),  
    ReportTime DATETIME,  
    ConditionScore FLOAT NOT NULL CHECK (ConditionScore BETWEEN 0 AND 5),  
    Description CLOB NOT NULL,  
    BikeID CHAR(4) NOT NULL,  
    MaintenanceID CHAR(4),  
    InspectorID CHAR(4),  
    PRIMARY KEY (IssueID),  
    FOREIGN KEY (ConditionScore, Description) REFERENCES  
IssueRule(ConditionScore, Description),  
    FOREIGN KEY (BikeID) REFERENCES Bike(BikeID),  
    FOREIGN KEY (MaintenanceID) REFERENCES Maintenance(MaintenanceID),  
    FOREIGN KEY (InspectorID) REFERENCES Inspector(StaffID)  
);
```

MaintenanceTask

```
CREATE TABLE MaintenanceTask (  
    TaskID CHAR(4),  
    MaintenanceID CHAR(4) NOT NULL,  
    Duration TIME,  
    StartTime DATETIME,  
    EndTime DATETIME,
```

```
TechnicianID CHAR(4),  
PRIMARY KEY (TaskID, MaintenanceID),  
FOREIGN KEY (MaintenanceID) REFERENCES Maintenance(MaintenanceID)  
ON DELETE CASCADE,  
FOREIGN KEY (TechnicianID) REFERENCES Technician(StaffID)  
);
```

Justification:

- MaintenanceTask is a weak entity that belongs to Maintenance, so it depends on Maintenance. If a maintenance record is removed, its corresponding tasks at MaintenanceTask should also be removed. So, we use ON DELETE CASCADE for the foreign key MaintenanceID.

Maintenance

```
CREATE TABLE Maintenance (  
MaintenanceID CHAR(4),  
PriorityLevel INTEGER CHECK (PriorityLevel >= 0),  
CompletionStatus VARCHAR(10) NOT NULL,  
RepairQuotation CLOB,  
PRIMARY KEY (MaintenanceID)  
);
```

Staff

```
CREATE TABLE Staff (  
StaffID CHAR(4),  
Name VARCHAR(30),  
Contact VARCHAR(20),  
AssignedArea CLOB,  
RoleID CHAR(4),  
ManagerID CHAR(4) NOT NULL,  
PRIMARY KEY (StaffID),  
FOREIGN KEY (ManagerID) REFERENCES Staff(StaffID)  
);
```

Technician

```
CREATE TABLE Technician (  
StaffID CHAR(4),  
PRIMARY KEY (StaffID),  
FOREIGN KEY (StaffID)  
REFERENCES Staff(StaffID) ON DELETE CASCADE  
);
```


Justification:

- Technician is a subtype of Staff, so we add ON DELETE CASCADE to ensure referential integrity because a subtype tuple cannot exist without the existence of its corresponding supertype tuple.

Inspector

```
CREATE TABLE Inspector (  
    StaffID CHAR(4),  
    PRIMARY KEY (StaffID),  
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID) ON DELETE CASCADE  
);
```

Justification:

- Inspector is a subtype of Staff, so we add ON DELETE CASCADE to ensure referential integrity because a subtype tuple cannot exist without the existence of its corresponding supertype tuple.

Manager

```
CREATE TABLE Manager (  
    StaffID CHAR(4),  
    PRIMARY KEY (StaffID),  
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID) ON DELETE CASCADE  
);
```

Justification:

- Manager is a subtype of Staff, so we add ON DELETE CASCADE to ensure referential integrity because a subtype tuple cannot exist without the existence of its corresponding supertype tuple.

Acknowledgement of Using AI Tools

Yes. AI tools (e.g., ChatGPT) were used for language polishing, clarification of written explanations, word choice refinement, and document formatting. The schema, functional dependencies, normalization, SQL DDL statements and all technical content were independently produced by the project team.